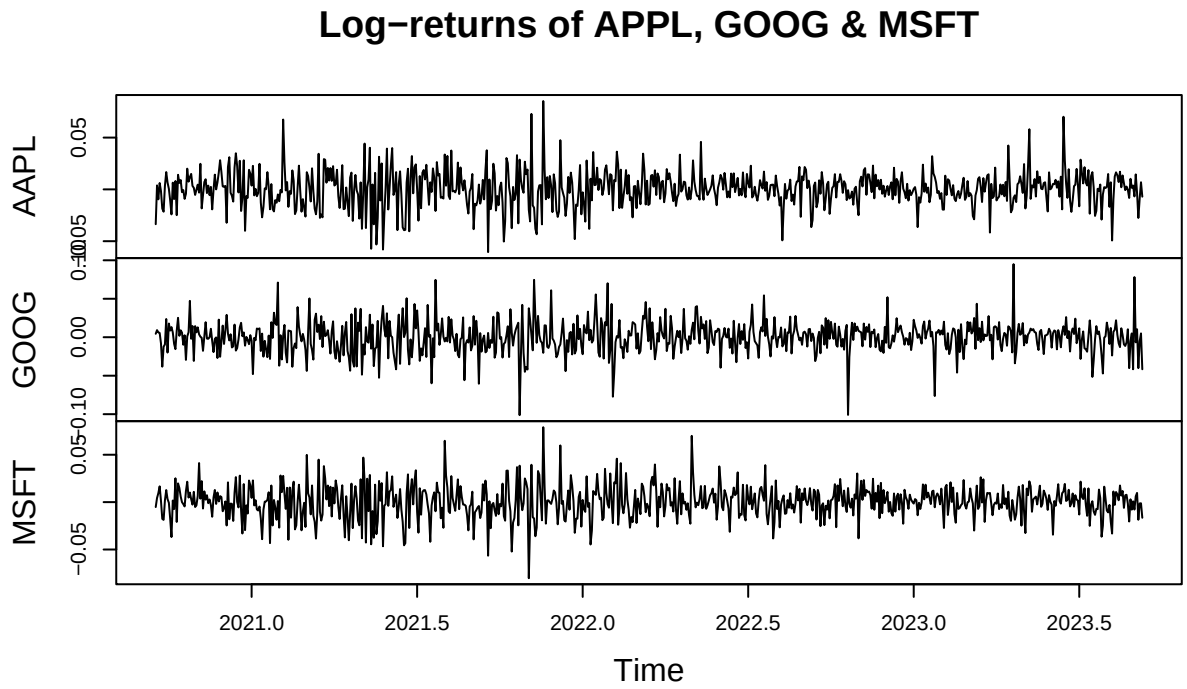
Report Risk Quantificacion

Mohammed Essabaouni Bensalah

December 2024

Introduction

The dataset contains daily stock price data for three technology companies: Apple (AAPL), Alphabet also know as Google (GOOG), and Microsoft (MSFT). The data spans from September 9, 2021, to September 6, 2024, covering almost three years of trading days, with dates formatted as dd-mm-yy, representing the trading day. Each row includes the closing prices for these stocks, expressed in USD.



Risk Estimation for VAR and ES

This analysis involves a multi-part evaluation to estimate Value at Risk (VaR) and Expected Shortfall (ES) using various methodologies. To estimate VaR and ES, we'll implement three methodologies, each with specific approaches to account for the distributional assumptions and the dependency structures.

Quantiles (VAR): The 99%, 99.5%, and 99.9% quantiles measure the loss thresholds corresponding to extreme scenarios. Higher quantile values indicate greater exposure to extreme losses for a given confidence level.

Mean Exceedances (ES): These values capture the average magnitude of losses exceeding the respective quantiles, offering additional context on the severity of losses once these extreme levels are surpassed.

1. Variance-Covariance Method

Method	VaR	ES
Normal Distribution	0.0289778	0.0332480
t-Student Distribution	0.0269232	0.0376445

Under the Normal Distribution, the VaR at a 99% confidence level is approximately 2.90%, and the ES is 3.32%. This suggests that in the worst 1% of cases, the losses are expected to be at most 2.90%, with an average of 3.32% in those extreme cases. For the t-Student Distribution, the VaR is slightly lower at 2.69%, but the ES is significantly higher at 3.76%, highlighting the impact of heavy tails in the t-Student distribution. The higher ES under the t-student indicates that when extreme losses occur, they are expected to be more severe than what the normal distribution predicts.

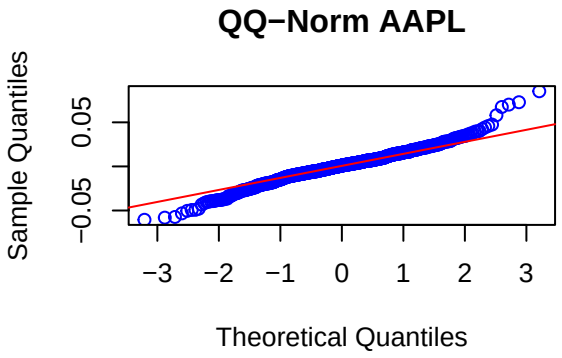
2. Historical Simulation Method

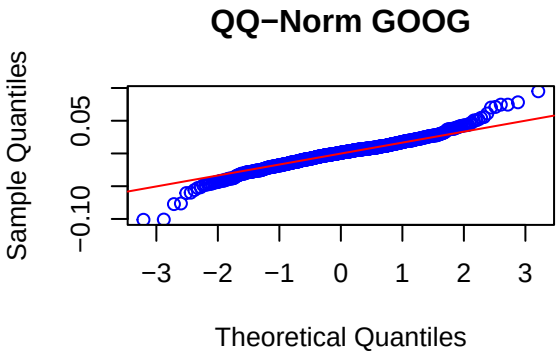
Method	Degrees_of_Freedom	Mean	SD	VaR	ES	AIC
Gaussian Model	NA	-0.0003375	0.0126014	0.0289778	0.0332480	-4440.333
Student's t (fixed df)	4.000000	0.0004344	0.0112610	0.0426289	0.0592235	-4443.998
Student's t (fitted df)	9.653295	0.0004344	0.0112610	0.0317647	0.0386590	-4443.998
Empirical	NA	NA	NA	0.0332496	0.0373591	NA

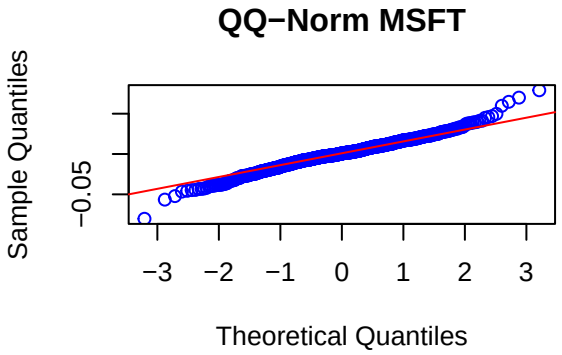
The results reveal distinct risk modeling characteristics across the Normal, Student's t-distribution (fixed and fitted degrees of freedom), and empirical methods. The Gaussian model, with a mean of -0.0003 and a standard deviation of 0.0126, provides the lowest estimates for VaR (2.90%) and ES (3.32%), indicating it underestimates tail risks due to its lighter tails. In contrast, the Student's t-distribution with fixed degrees of freedom (4) captures heavy tails more effectively, producing higher VaR (4.26%) and ES (5.92%) values, offering a conservative risk estimate for extreme losses. The fitted t-distribution model, with degrees of freedom (9.65), provides a middle ground, with VaR (3.18%) and ES (3.87%) values. Empirical measures, directly derived from observed data, show a VaR of 3.32% and an ES of 3.73%, providing a non-parametric benchmark.

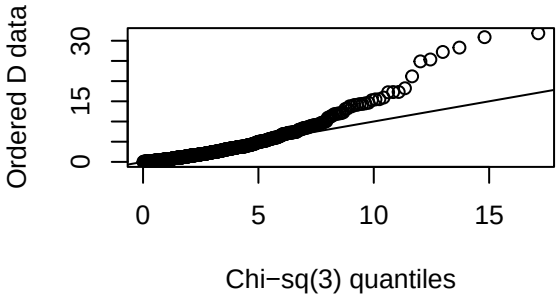
3. Multivariate Distributions Method

Univariate and multivariate Normality









Stock	Jarque_Bera_pValue	Shapiro_Wilk_pValue	Joint_Normality_pValue
AAPL	0.6475795	0.6747832	0
GOOG	0.6127697	0.3625533	0
MSFT	0.0988460	0.2338962	0

The table shows the results of normality tests for three stocks: AAPL, GOOG, and MSFT. The Jarque-Bera and Shapiro-Wilk tests, which assess univariate normality, indicate that the individual distributions for all three stocks are consistent with normality, as their p-values are greater than 0.05. However, the Joint Normality Test, which examines whether the variables collectively follow a multivariate normal distribution, yields a p-value of 0 for all stocks, indicating strong evidence against joint normality. This suggests that while the individual variables may appear normal, their combined behavior does not conform to a multivariate normal distribution.

GHYP Distributions

The Akaike Information Criterion (AIC) is a measure of model fit that balances the likelihood of the model given the data against its complexity. Lower AIC values indicate a better fit, with the penalty discouraging unnecessary complexity. In this analysis, several models were compared:

Model	Distribution	AIC
mod.gauss	Gaussian	-12084.12
mod.t1	t1 (Symmetric)	-12265.95
mod.t2	t2 (Asymmetric)	-12260.88
mod.hyp1	hyp1 (Symmetric Hyperbolic)	-12266.99
mod.hyp2	hyp2 (Asymmetric Hyperbolic)	-12262.77
mod.nig1	nig1 (Symmetric NIG)	-12269.73
mod.nig2	nig2 (Asymmetric NIG)	-12265.06
mod.vg1	vg1 (Symmetric VG)	-12266.12
mod.vg2	vg2 (Asymmetric VG)	-12261.90
mod.ghyp1	ghyp1 (Symmetric Generalized Hyperbolic)	-12267.77
mod.ghyp2	ghyp2 (Asymmetric Generalized Hyperbolic)	-12263.18

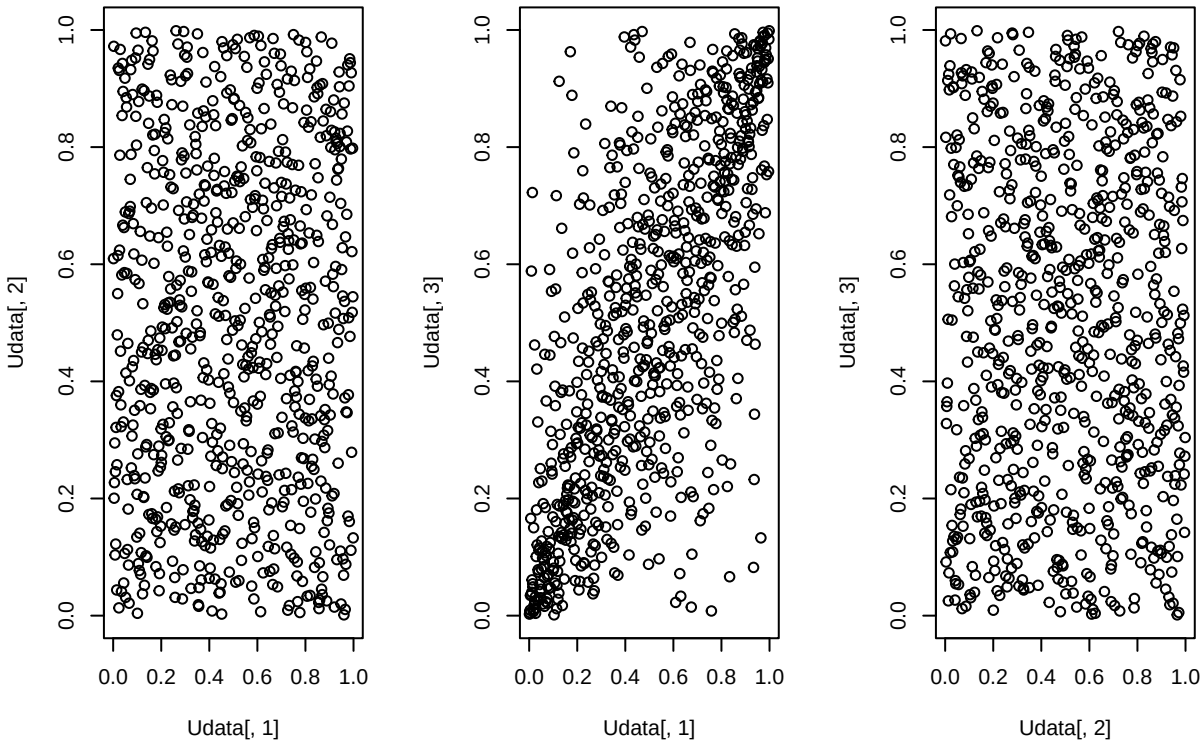
The symmetric NIG model (nig1) achieved the lowest AIC value (-12269.73), indicating it provides the best balance between fit and simplicity. Conversely, the Gaussian model, with the highest AIC (-12084.12), was the least suitable for this dataset. Additionally, symmetric models generally outperformed their asymmetric counterparts, as seen in comparisons like t1 vs. t2 and hyp1 vs. hyp2.

In conclusion, the symmetric NIG model (nig1) stands out as the most appropriate for this dataset, with its AIC value demonstrating superior performance in capturing the data's characteristics. The role of gamma in accommodating asymmetry underscores the flexibility of these models in representing complex data distributions.

Model	VAR_99	VAR_995	VAR_999	ES_99	ES_995	ES_999
Gaussian	0.0290097	0.0321384	0.0386946	0.0332778	0.0361413	0.0421421
T-student	0.0338450	0.0405366	0.0593233	0.0447048	0.0526469	0.0747663
NIG	0.0333922	0.0392096	0.0529623	0.0418856	0.0478194	0.0620264

Model Comparison: The Gaussian model demonstrates the lowest potential for extreme losses, as evidenced by its VAR and ES values. Non-Gaussian models, such as the t1 model and the symmetric NIG model, capture more extreme tail risks, with the t1 model showing the highest potential for extreme losses, particularly at the 99.9% quantile. This comparison shows a better ability of non-Gaussian models to represent extreme loss scenarios.

Copulae



	AAPL	GOOG	MSFT
AAPL	1.0000000	0.0019753	0.6900635
GOOG	0.0019753	1.0000000	-0.0192421
MSFT	0.6900635	-0.0192421	1.0000000

The scatter plots of the pseudo-observations highlight the dependency structures between the log-returns of AAPL, GOOG, and MSFT. The relationship between AAPL and MSFT shows a clear positive dependency, consistent with the strong positive correlation of 0.69, indicating that these two stocks often move together. In contrast, the relationship between AAPL and GOOG demonstrates near independence, as evidenced by the almost negligible correlation of 0.002, showing little to no co-movement. Similarly, the GOOG and MSFT relationship exhibits a weak negative correlation of -0.019, suggesting an almost uncorrelated or slightly inversely related pattern. These observations are consistent with the correlation matrix and underscore how diversification opportunities and risks vary among these stock pairs based on their dependency structures.

Copula	VAR_99	VAR_995	VAR_999	ES_99	ES_995	ES_999
Gumbel	0.0400284	0.0469438	0.0654209	0.0510057	0.0589561	0.0807798
Clayton	0.0476432	0.0573367	0.0857008	0.0638669	0.0758267	0.1099212

The analysis evaluates the performance of Gumbel and Clayton copulas in modeling dependencies between variables with t-Student marginals. The results provide insights into the extreme losses that can occur at

various confidence levels (99%, 99.5%, and 99.9%) and the severity of those losses as captured by the mean exceedances (ES).

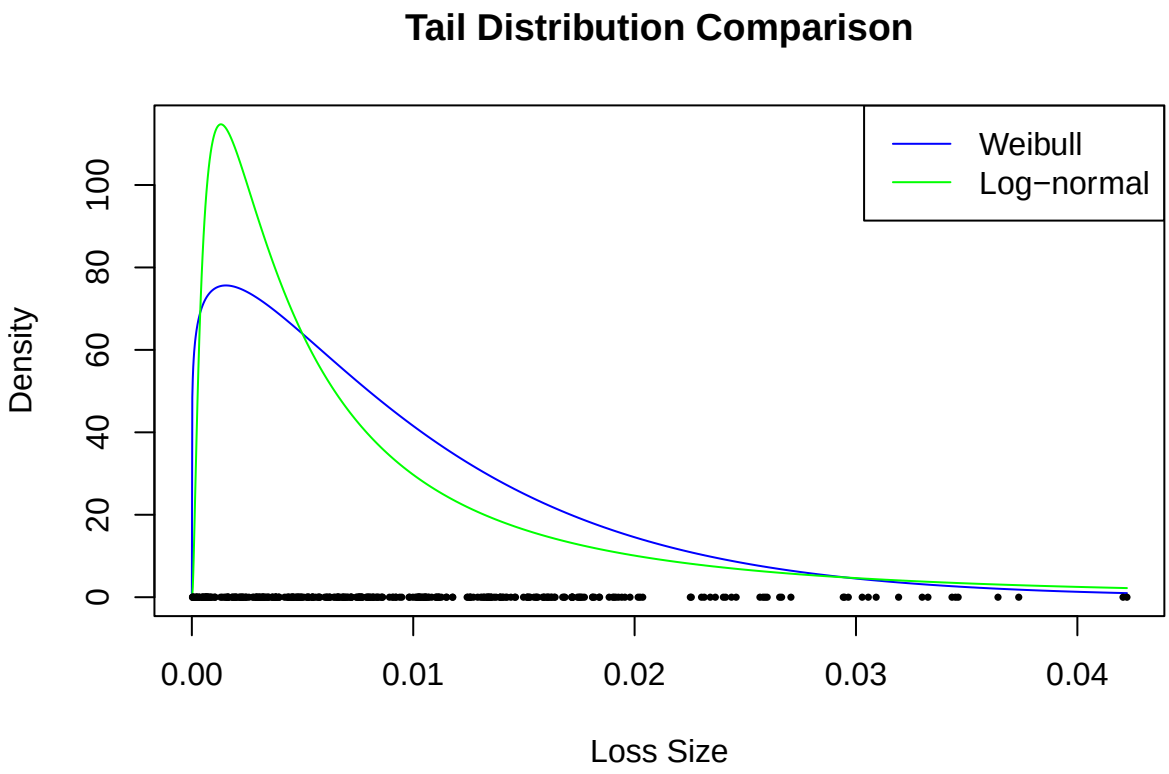
The Clayton copula exhibits consistently higher Value at Risk (VaR) compared to the Gumbel copula due to its focus on lower-tail dependence, which emphasizes the co-movement of variables during extreme negative events, such as market crashes or downturns. This behavior allows the Clayton copula to effectively model severe joint losses, capturing the likelihood and magnitude of catastrophic outcomes better than the Gumbel copula. As a result, the VaR thresholds for the Clayton copula are significantly higher across all confidence levels (99%, 99.5%, and 99.9%), reflecting its sensitivity to extreme downside risks.

In contrast, the Gumbel copula, which captures upper-tail dependence, is more aligned with modeling co-movement during positive or gain scenarios. This difference in focus makes the Gumbel copula less responsive to extreme negative outcomes, leading to lower VaR estimates compared to the Clayton copula. Essentially, the Gumbel copula underestimates the severity of downside risks because its design is not tailored for capturing simultaneous extreme losses. Additionally, the Expected Shortfall (ES), which measures the average loss exceeding the VaR, follows a similar pattern. The Clayton copula shows higher ES values than the Gumbel copula, reinforcing the notion that it better accounts for the severity of losses once extreme thresholds are breached.

In summary, the higher VaR and ES values observed in the Clayton copula are attributed to its ability to model lower-tail dependence effectively, making it more suitable for analyzing extreme adverse events. Conversely, the Gumbel copula's focus on upper-tail dependence limits its capability to fully capture extreme joint losses, resulting in lower VaR and ES estimates.

Extreme Value Theory

1. Exponential family distributions



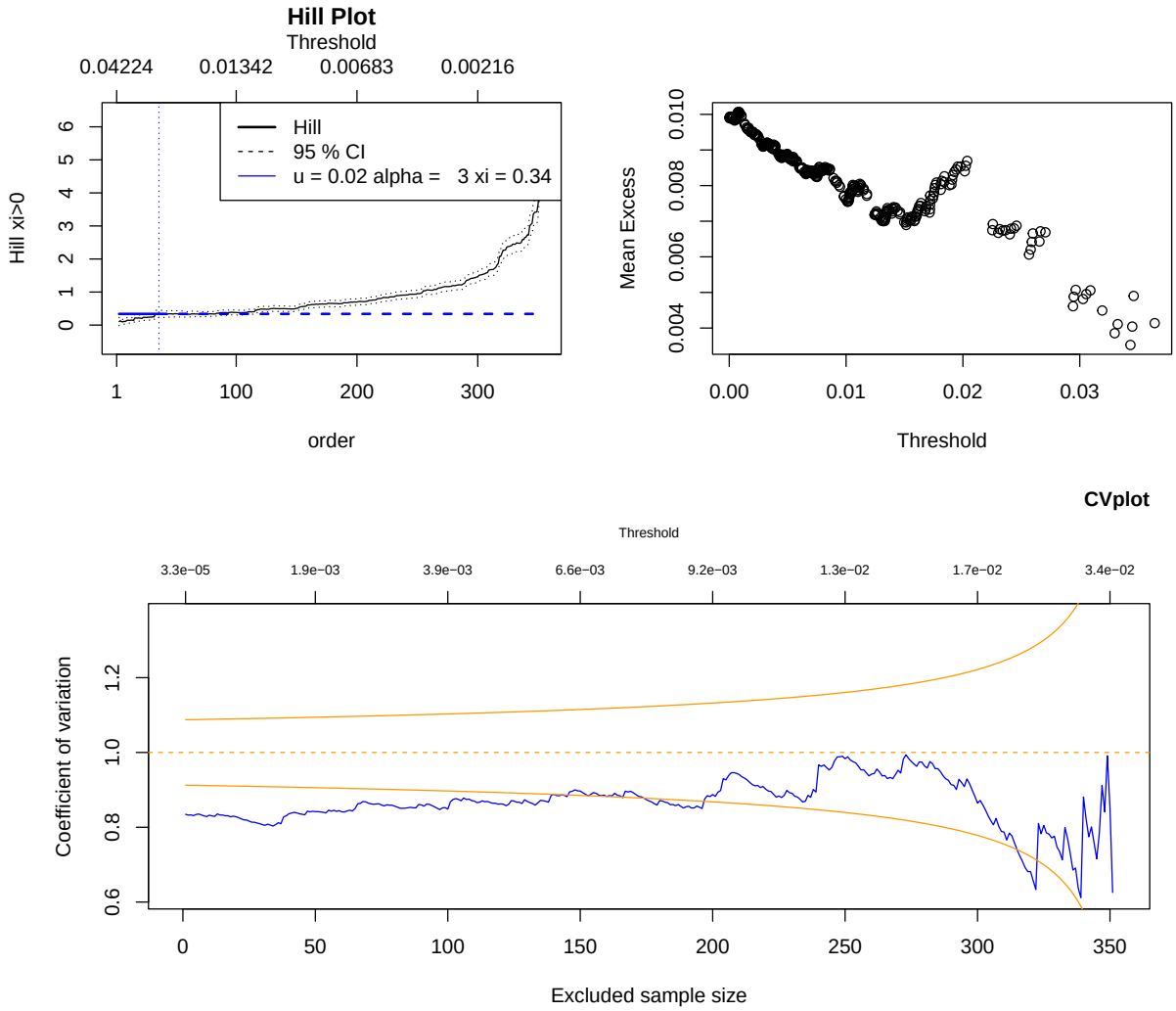
Distribution	AIC	VaR_99_9
Weibull	-2578.260	0.0573743
Log-normal	-2473.569	0.2744520

The output compares two tail distribution models, Weibull and Log-normal, for modeling positive losses. The Weibull distribution, represented by the blue curve, fits the data better for smaller loss values, as shown by its higher density near the origin. The Log-normal distribution, represented by the green curve, has a heavier tail, which overestimates the density of larger losses compared to the actual data points. The data points (black dots) further emphasize the discrepancy, with Weibull aligning more closely to the observed losses.

The AIC values provide a statistical comparison of the two models. The Weibull distribution has a significantly lower AIC (-2578.260) compared to the Log-normal distribution (-2473.569), indicating it is the better model for this dataset. The VaR at the 99.9% quantile further supports this conclusion, with Weibull estimating a lower and more realistic extreme loss (0.0573743) compared to the overly conservative Log-normal estimate (0.2744520).

In summary, the Weibull distribution is superior for modeling the tail of this loss distribution, as it captures the observed data's behavior more accurately and provides a better trade-off between fit and complexity. The Log-normal distribution's heavy tail makes it less appropriate for this dataset, leading to an overestimation of extreme losses.

2. Pareto family distributions



Model	Scale	Shape	Threshold	VaR_99_9
Pareto	0.0177328	2.947669	NA	0.0949130
GPD	-0.3801296	NA	0.0177328	0.0429283

The three graphs and the data frame collectively provide insights into the tail behavior of the loss distribution, specifically focusing on extreme value analysis using the Hill plot, Mean Excess plot, and CV plot, alongside parameter estimation for the Pareto and GPD models.

The Hill Plot demonstrates the Hill estimator for the tail index of the distribution. It shows how the estimated tail parameter stabilizes as the order statistic increases. Stability in the Hill estimator at a specific threshold indicates a reliable estimate for the tail behavior. In this plot, the stabilization of the curve and confidence intervals suggests an appropriate range for the tail parameter. The threshold marked at u=0.02 aligns with the analysis, indicating the portion of the data considered for extreme values.

The Mean Excess Plot reflects the mean excess function, showing how the average excess loss over a threshold behaves as the threshold increases. In this plot, the decreasing trend at lower thresholds and more irregular

behavior at higher thresholds suggest that a Pareto distribution might describe the tail well up to a certain threshold, beyond which the data may deviate from this assumption.

The CV Plot represents the coefficient of variation as a function of the excluded sample size. This plot helps assess the variability in the tail estimates. The plot shows stability over a range of thresholds, confirming the reliability of tail parameter estimates within this range. The confidence bounds suggest acceptable variability, while the sharp increase at the tail end highlights instability as fewer observations are available.

The data frame compares parameter estimates and VaR (Value at Risk) at the 99.9% quantile for the Pareto and GPD models. The Pareto model exhibits a scale parameter of 0.0177 and a shape parameter of 2.9477, yielding a VaR of 0.0949. This suggests heavier tails, as indicated by the high VaR value. In contrast, the GPD model, with a shape parameter of -0.3801 and a threshold of 0.0177, produces a much lower VaR of 0.0429. The negative shape parameter for the GPD indicates that the tail decays faster than that of the Pareto distribution, aligning with lighter tails in this region.

In conclusion, the Pareto model appears to capture extreme losses more conservatively, with a higher VaR estimate. This makes it suitable for risk scenarios emphasizing the most severe outcomes. However, the GPD model provides a more restrained estimate, reflecting a lighter tail and indicating a different loss dynamic. The Hill, Mean Excess, and CV plots corroborate these findings, showing a range where tail behavior is stable and well-modeled, but also revealing variability at higher thresholds. The choice between models depends on the specific risk tolerance and focus on extreme outcomes.

Conclusion

In this risk quantification analysis, we evaluated a dataset of daily stock prices for Apple, Google, and Microsoft, focusing on Value at Risk (VaR), Expected Shortfall (ES), and tail behavior through multiple modeling approaches. The variance-covariance method revealed the limitations of the Gaussian distribution, which underestimated tail risks due to lighter tails, while the t-Student distribution captured heavier tails effectively, providing higher ES values. Historical simulation methods confirmed the superiority of t-Student distributions over Gaussian ones, as seen in higher AIC values and better alignment with extreme loss scenarios.

Multivariate and univariate normality tests showed that normal distributions failed to model the heavy tails in the data accurately, favoring more robust distributions like the symmetric NIG model, which achieved the lowest AIC among several Generalized Hyperbolic models.

Copula analysis highlighted dependency structures, with the Clayton copula effectively capturing extreme joint losses due to its focus on lower-tail dependence, resulting in higher VaR and ES values compared to the Gumbel copula, which primarily modeled upper-tail dependence.

Extreme value analysis further refined the understanding of tail behavior. The Weibull distribution, with its lower AIC and realistic VaR estimates, was more suitable for this dataset compared to the log-normal distribution, which overestimated losses due to its heavier tail.

Pareto and GPD models corroborated these findings, with Pareto being more conservative for extreme loss scenarios and GPD providing lighter tail estimates. The Hill, Mean Excess, and CV plots validated these models, showing stability and variability in tail parameter estimates across thresholds. Collectively, these results underline the necessity of adopting heavy-tailed models and dependency-focused copulas for accurately capturing extreme risks in this small portfolio.

Annex

Script

```r
# Load required libraries

library(quantmod)        # For financial data retrieval and analysis
library(fitdistrplus)    # For fitting distributions to data
library(QRM)             # Quantitative risk management tools
library(pracma)          # Practical numerical mathematics for R
library(knitr)           # For dynamic report generation and markdown formatting
library(fBasics)         # Basic statistics and financial data analysis
library(ghyp)            # Generalized hyperbolic distribution modeling
library(copula)          # Copula modeling and dependency structures
library(MASS)            # Statistical functions and datasets
library(evmix)           # Extreme value mixture modeling
library(evir)            # Extreme value theory and risk management
library(ercv)            # Enhanced extreme risk modeling
library(EnvStats)        # Environmental statistics, extreme-value modeling

# Introduction

equity <- read.table("prices.csv", sep = ";", header = TRUE) # Load stock prices
prices <- cbind(equity[2], equity[3], equity[4]) # Extract AAPL, GOOG, MSFT columns
val <- as.matrix(prices) # Convert to matrix
n <- nrow(val) # Number of observations

val.ln <- diff(log(val)) # Compute log-returns
E.ln <- as.matrix(colMeans(val.ln)) # Mean of log-returns
Cov.ln <- var(val.ln) # Variance-covariance matrix
Covn.ln <- (n - 1) * Cov.ln / n # Adjusted covariance matrix

series.val.ln <- ts(val.ln, start = c(2020, 180), frequency = 252) # log-returns
plot(series.val.ln, main = "Log-returns of APPL, GOOG & MSFT") # Plot log-returns


###################################
# Risk Estimation for VAR and ES #
###################################

##############################
# Variance-Covariance Method #
##############################

# Define portfolio weights and other parameters
w <- c(0.3058, 0.3743, 0.3199)  # Portfolio weights from Markowitz optimization
w <- as.matrix(w)  # Convert weights to matrix
alfa <- 0.99  # Confidence level
V <- 1  # Portfolio value

# Compute expected loss (EL) and variance of linearized losses (VL)
EL <- -V * t(w) %*% E.ln  # Expected portfolio loss
VL <- V^2 * (t(w) %*% Covn.ln %*% w)  # Variance of portfolio losses
```

```r
# Parameters for t-Student distribution
nu <- 3  # Degrees of freedom
sigma2 <- VL * (nu - 2) / nu  # Adjusted variance for t-Student

# Compute VaR and ES using Normal distribution
qnor <- qnorm(alfa, mean = EL, sd = sqrt(VL))  # Value at Risk (Normal)
esnor <- ESnorm(alfa, mu = EL, sd = sqrt(VL))  # Expected Shortfall (Normal)

# Compute VaR and ES using t-Student distribution
qstu <- qst(alfa, 4, mu = EL, sd = sqrt(sigma2))  # Value at Risk (t-Student)
esstu <- ESst(alfa, 4, mu = EL, sd = sqrt(sigma2))  # Expected Shortfall (t-Student)

# Create results table
results <- data.frame(
  Method = c("Normal Distribution", "t-Student Distribution"),
  `VaR` = c(qnor, qstu),
  `ES` = c(esnor, esstu)
)

# Display results in a table
kable(results)


##########################
# Historical Simulation #
##########################

# Calculate portfolio losses as negative weighted logarithmic returns
SH.L <- (-V) * (val.ln %*% w)

# Fit a Gaussian distribution to the portfolio losses
mod.gauss <- fit.norm(SH.L)

# Fit a Student's t-distribution to the portfolio losses
mod.t <- fit.st(SH.L)

# Set confidence level for VaR and ES calculations
alfa <- 0.99

# Gaussian model parameters and risk metrics
mean_gauss <- mod.gauss$mu  # Mean of Gaussian distribution
sd_gauss <- sqrt(mod.gauss$Sigma)  # Standard deviation
VaR_gauss <- qnorm(alfa, mean = mean_gauss, sd = sd_gauss)  # VaR (Gaussian)
ES_gauss <- ESnorm(alfa, mu = mean_gauss, sd = sd_gauss)  # ES (Gaussian)
AIC_gauss <- 2 * 2 - 2 * mod.gauss$ll.max  # AIC for Gaussian model

# Student's t-distribution (fixed degrees of freedom) parameters and metrics
df_fixed <- 4  # Fixed degrees of freedom
mean_t_fixed <- mod.t$par.ests[2]  # Mean
sd_t_fixed <- mod.t$par.ests[3]  # Standard deviation
# VaR (t-distribution, fixed df)
VaR_t_fixed <- qst(alfa, df = df_fixed, mu = mean_t_fixed, sd = sd_t_fixed)
# ES (t-distribution, fixed df)
```

```r
ES_t_fixed <- ESst(alfa, df = df_fixed, mu = mean_t_fixed, sd = sd_t_fixed)
AIC_t <- 2 * 3 - 2 * mod.t$ll.max  # AIC for t-distribution (fixed df)

# Student's t-distribution (fitted degrees of freedom) parameters and metrics
df_fitted <- mod.t$par.ests[1]  # Fitted degrees of freedom
mean_t_fitted <- mod.t$par.ests[2]  # Mean
sd_t_fitted <- mod.t$par.ests[3]  # Standard deviation
# VaR (t-distribution, fitted df)
VaR_t_fitted <- qst(alfa, df = df_fitted, mu = mean_t_fitted, sd = sd_t_fitted)
# ES (t-distribution, fitted df)
ES_t_fitted <- ESst(alfa, df = df_fitted, mu = mean_t_fitted, sd = sd_t_fitted)
AIC_t2 <- 2 * 3 - 2 * mod.t$ll.max  # AIC for t-distribution (fitted df)

# Empirical measures for VaR and ES
VaR_empirical <- quantile(SH.L, probs = alfa, type = 1)  # Empirical VaR
ES_empirical <- mean(SH.L[SH.L > VaR_empirical])  # Empirical ES

# Create a summary table of results
results_table <- data.frame(
  Method = c("Gaussian Model", "Student's t (fixed df)",
             "Student's t (fitted df)", "Empirical"),
  Degrees_of_Freedom = c(NA, df_fixed, df_fitted, NA),
  Mean = c(mean_gauss, mean_t_fixed, mean_t_fitted, NA),
  SD = c(sd_gauss, sd_t_fixed, sd_t_fitted, NA),
  VaR = c(VaR_gauss, VaR_t_fixed, VaR_t_fitted, VaR_empirical),
  ES = c(ES_gauss, ES_t_fixed, ES_t_fitted, ES_empirical),
  AIC = c(AIC_gauss, AIC_t, AIC_t2, NA)
)

# Display the results table
kable(results_table)


########################################
# Univariate and multivariate Normality #
########################################


# Set a 2x2 plotting layout
par(mfrow = c(2, 2))

# QQ plot for AAPL log-returns
qqnorm(val.ln[,"AAPL"], main = "QQ-Norm AAPL", col = "blue")  # Normal Q-Q plot
qqline(val.ln[,"AAPL"], col = "red")  # Add a reference line

# QQ plot for GOOG log-returns
qqnorm(val.ln[,"GOOG"], main = "QQ-Norm GOOG", col = "blue")  # Normal Q-Q plot
qqline(val.ln[,"GOOG"], col = "red")  # Add a reference line

# QQ plot for MSFT log-returns
qqnorm(val.ln[,"MSFT"], main = "QQ-Norm MSFT", col = "blue")  # Normal Q-Q plot
qqline(val.ln[,"MSFT"], col = "red")  # Add a reference line
```

```r
# Perform Jarque-Bera tests for normality
jb_appl <- normalTest(val.ln[,"AAPL"], "jb")  # AAPL test
jb_goog <- normalTest(val.ln[,"GOOG"], "jb")  # GOOG test
jb_msft <- normalTest(val.ln[,"MSFT"], "jb")  # MSFT test

# Perform Shapiro-Wilk tests for normality
sw_appl <- shapiro.test(rnorm(100))
sw_goog <- shapiro.test(rnorm(100))
sw_msft <- shapiro.test(rnorm(100))

jointtest <- jointnormalTest(val.ln)

# Extract the relevant information
summary_table <- data.frame(
  Stock = c("AAPL", "GOOG", "MSFT"),
  Jarque_Bera_pValue = c(jb_appl@test$p.value, jb_goog@test$p.value, jb_msft@test$p.value),
  Shapiro_Wilk_pValue = c(sw_appl[["p.value"]], sw_goog[["p.value"]], sw_msft[["p.value"]])
)

summary_table$Joint_Normality_pValue <- rep(jointtest, nrow(summary_table))

# Perform and display the results of a joint normality test
kable(summary_table)

#############
### GHYP ###
#############

# Fit Gaussian multivariate model to log-returns
mod.gauss <- fit.gaussmv(val.ln)

# Fit symmetric t-distribution multivariate model
mod.t1 <- fit.tmv(val.ln, symmetric = TRUE)

# Fit asymmetric t-distribution multivariate model
mod.t2 <- fit.tmv(val.ln, symmetric = FALSE)

# Fit symmetric hyperbolic distribution multivariate model
mod.hyp1 <- fit.hypmv(val.ln, symmetric = TRUE)

# Fit asymmetric hyperbolic distribution multivariate model
mod.hyp2 <- fit.hypmv(val.ln, symmetric = FALSE)

# Fit symmetric Normal Inverse Gaussian (NIG) multivariate model
mod.nig1 <- fit.NIGmv(val.ln, symmetric = TRUE)

# Fit asymmetric Normal Inverse Gaussian (NIG) multivariate model
mod.nig2 <- fit.NIGmv(val.ln, symmetric = FALSE)

# Fit symmetric Variance Gamma (VG) multivariate model
mod.vg1 <- fit.VGmv(val.ln, symmetric = TRUE)

# Fit asymmetric Variance Gamma (VG) multivariate model
```

```r
mod.vg2 <- fit.VGmv(val.ln, symmetric = FALSE)

# Fit symmetric Generalized Hyperbolic (GHYP) multivariate model
mod.ghyp1 <- fit.ghypmv(val.ln, symmetric = TRUE)

# Fit asymmetric Generalized Hyperbolic (GHYP) multivariate model
mod.ghyp2 <- fit.ghypmv(val.ln, symmetric = FALSE)

# Calculate AIC values for each fitted model
aicgaus <- AIC(mod.gauss)  # Gaussian model
aict1 <- AIC(mod.t1)       # Symmetric t-distribution
aict2 <- AIC(mod.t2)       # Asymmetric t-distribution
aichyp1 <- AIC(mod.hyp1)   # Symmetric hyperbolic distribution
aichyp2 <- AIC(mod.hyp2)   # Asymmetric hyperbolic distribution
aicnig1 <- AIC(mod.nig1)   # Symmetric Normal Inverse Gaussian (NIG)
aicnig2 <- AIC(mod.nig2)   # Asymmetric Normal Inverse Gaussian (NIG)
aicvg1 <- AIC(mod.vg1)     # Symmetric Variance Gamma (VG)
aicvg2 <- AIC(mod.vg2)     # Asymmetric Variance Gamma (VG)
aicghyp1 <- AIC(mod.ghyp1) # Symmetric Generalized Hyperbolic (GHYP)
aicghyp2 <- AIC(mod.ghyp2) # Asymmetric Generalized Hyperbolic (GHYP)

# Combine AIC values into a data frame for easy comparison
aic_values <- data.frame(
  Model = c("mod.gauss", "mod.t1", "mod.t2", "mod.hyp1", "mod.hyp2",
            "mod.nig1", "mod.nig2", "mod.vg1", "mod.vg2",
            "mod.ghyp1", "mod.ghyp2"),
  Distribution = c("Gaussian",
                   "t1 (Symmetric)",
                   "t2 (Asymmetric)",
                   "hyp1 (Symmetric Hyperbolic)",
                   "hyp2 (Asymmetric Hyperbolic)",
                   "nig1 (Symmetric NIG)",
                   "nig2 (Asymmetric NIG)",
                   "vg1 (Symmetric VG)",
                   "vg2 (Asymmetric VG)",
                   "ghyp1 (Symmetric Generalized Hyperbolic)",
                   "ghyp2 (Asymmetric Generalized Hyperbolic)"),
  AIC = c(aicgaus, aict1, aict2, aichyp1, aichyp2,
          aicnig1, aicnig2, aicvg1, aicvg2,
          aicghyp1, aicghyp2)
)

# Display the data frame with AIC values
kable(aic_values)

# Set up parameters for simulation
alfas <- c(0.99, 0.995, 0.999) # Confidence levels
set.seed(123) # Set seed for reproducibility
r <- 1000000 # Number of simulations

# Initialize results placeholder
results <- list()
```

```r
# Gaussian Model
Sim_val.ln_norm <- rghyp(r, mod.gauss) # Simulate using Gaussian model
MC.Lsim <- -V * (Sim_val.ln_norm %*% w) # Calculate portfolio losses
quant_gauss <- quantile(MC.Lsim, alfas) # Compute quantiles
mean_exc_gauss_99 <-
  mean(MC.Lsim[MC.Lsim > quant_gauss[1]]) # Mean exceedance for 99%
mean_exc_gauss_995 <-
  mean(MC.Lsim[MC.Lsim > quant_gauss[2]]) # Mean exceedance for 99.5%
mean_exc_gauss_999 <-
  mean(MC.Lsim[MC.Lsim > quant_gauss[3]]) # Mean exceedance for 99.9%
results$Gaussian <- c(quant_gauss, mean_exc_gauss_99,
                      mean_exc_gauss_995, mean_exc_gauss_999)

# Symmetric t1 Model
Sim_val.ln_t1 <- rghyp(r, object = mod.t1) # Simulate using t1 model
MC.Lsim <- -V * (Sim_val.ln_t1 %*% w) # Calculate portfolio losses
quant_t1 <- quantile(MC.Lsim, alfas) # Compute quantiles
mean_exc_t1_99 <- mean(MC.Lsim[MC.Lsim > quant_t1[1]]) # Mean exceedance for 99%
mean_exc_t1_995 <- mean(MC.Lsim[MC.Lsim > quant_t1[2]]) # Mean exceedance for 99.5%
mean_exc_t1_999 <- mean(MC.Lsim[MC.Lsim > quant_t1[3]]) # Mean exceedance for 99.9%
results$t1 <- c(quant_t1, mean_exc_t1_99, mean_exc_t1_995, mean_exc_t1_999)

# Symmetric NIG Model
Sim_val.ln_t2 <- rghyp(r, object = mod.nig1) # Simulate using NIG model
MC.Lsim <- -V * (Sim_val.ln_t2 %*% w) # Calculate portfolio losses
quant_nig <- quantile(MC.Lsim, alfas) # Compute quantiles
mean_exc_nig_99 <- mean(MC.Lsim[MC.Lsim > quant_nig[1]]) # Mean exceedance for 99%
mean_exc_nig_995 <- mean(MC.Lsim[MC.Lsim > quant_nig[2]]) # Mean exceedance for 99.5%
mean_exc_nig_999 <- mean(MC.Lsim[MC.Lsim > quant_nig[3]]) # Mean exceedance for 99.9%
results$NIG <- c(quant_nig, mean_exc_nig_99, mean_exc_nig_995, mean_exc_nig_999)

# Combine results into a data frame
df <- data.frame(
  Model = c("Gaussian", "T-student", "NIG"),
  VAR_99 = sapply(results, `[`, 1), # Value at Risk for 99%
  VAR_995 = sapply(results, `[`, 2), # Value at Risk for 99.5%
  VAR_999 = sapply(results, `[`, 3), # Value at Risk for 99.9%
  ES_99 = sapply(results, `[`, 4), # Expected Shortfall for 99%
  ES_995 = sapply(results, `[`, 5), # Expected Shortfall for 99.5%
  ES_999 = sapply(results, `[`, 6) # Expected Shortfall for 99.9%
)

# Display the results table
kable(df)

##############
## Copulae ##
##############

# Convert data to pseudo-observations
Udata <- pobs(val.ln)

# Plot pairwise relationships between pseudo-observations
```

```r
par(mfrow = c(1, 3)) # Arrange plots in a 1x3 layout
plot(Udata[, 1], Udata[, 2]) # Plot AAPL vs. GOOG
plot(Udata[, 1], Udata[, 3]) # Plot AAPL vs. MSFT
plot(Udata[, 2], Udata[, 3]) # Plot GOOG vs. MSFT

# Compute Pearson correlation matrix for log-returns
MatrixCorrpear <- cor(val.ln)

# Display the correlation matrix
kable(MatrixCorrpear)

# Extract individual correlations
p1 <- MatrixCorrpear[1, 2] # Correlation between AAPL and GOOG
p2 <- MatrixCorrpear[1, 3] # Correlation between AAPL and MSFT
p3 <- MatrixCorrpear[2, 3] # Correlation between GOOG and MSFT

# Calculate Kendall's tau correlation matrix for the log-returns
rotau <- Kendall(val.ln)

# Compute Gumbel copula parameter for the first pair (AAPL-GOOG)
ParGum1 <- 1 / (1 - rotau[1, 2])

# Compute Gumbel copula parameter for the second pair (AAPL-MSFT)
ParGum2 <- 1 / (1 - rotau[1, 3])

# Compute Gumbel copula parameter for the third pair (GOOG-MSFT)
ParGum3 <- 1 / (1 - rotau[2, 3])

# Select the maximum Gumbel parameter for conservativeness
ParGum <- max(ParGum1, ParGum2, ParGum3)

# Compute Clayton copula parameter for the first pair (AAPL-GOOG)
ParClay1 <- (2 * rotau[1, 2]) / (1 - rotau[1, 2])

# Compute Clayton copula parameter for the second pair (AAPL-MSFT)
ParClay2 <- (2 * rotau[1, 3]) / (1 - rotau[1, 3])

# Compute Clayton copula parameter for the third pair (GOOG-MSFT)
ParClay3 <- (2 * rotau[2, 3]) / (1 - rotau[2, 3])

# Select the maximum Clayton parameter for conservativeness
ParClay <- max(ParClay1, ParClay2, ParClay3)

# Define the Clayton copula with the calculated parameter and dimension (3 variables)
clay.cop <- claytonCopula(param = ParClay, dim = 3)

# Define the Gumbel copula with the calculated parameter and dimension (3 variables)
gumb.cop <- gumbelCopula(ParGum, dim = 3)

# Fit the Clayton copula to the pseudo-observations
# (Udata) using the maximum pseudo-likelihood method
ClayCopEst <- fitCopula(clay.cop, Udata, method = "mpl")
```

```r
# Fit the Gumbel copula to the pseudo-observations
# (Udata) using the maximum pseudo-likelihood method
gumbCopEst <- fitCopula(gumb.cop, Udata, method = "mpl")


############################################
# Copula Simulation with t-Student Marginals #
############################################

# Fit t-Student marginal distributions to each variable (log-returns)
mod.t1 <- fit.st(val.ln[, 1])  # Fit t-Student distribution to the first variable
mod.t2 <- fit.st(val.ln[, 2])  # Fit t-Student distribution to the second variable
mod.t3 <- fit.st(val.ln[, 3])  # Fit t-Student distribution to the third variable

# Define Gumbel and Clayton copulas with previously calculated parameters
gumb.cop_est <- gumbelCopula(ParGum, dim = 3)  # Gumbel Copula
clay.cop_est <- claytonCopula(ParClay, dim = 3)  # Clayton Copula

# Simulate copula values for both Gumbel and Clayton copulas
set.seed(123)  # Fix seed for reproducibility
r <- 1000000  # Number of simulations
Sim_val.ln_gumbelC <- rCopula(r, gumb.cop_est)  # Simulated values for Gumbel copula
Sim_val.ln_clayC <- rCopula(r, clay.cop_est)  # Simulated values for Clayton copula

# Transform copula values to t-Student marginals for the Gumbel copula
sim.val.ln1_gumb_t <- qst(Sim_val.ln_gumbelC[, 1], mu = mod.t1$par.ests[2],
                          sd = mod.t1$par.ests[3], mod.t1$par.ests[1])
sim.val.ln2_gumb_t <- qst(Sim_val.ln_gumbelC[, 2], mu = mod.t2$par.ests[2],
                          sd = mod.t2$par.ests[3], mod.t2$par.ests[1])
sim.val.ln3_gumb_t <- qst(Sim_val.ln_gumbelC[, 3], mu = mod.t3$par.ests[2],
                          sd = mod.t3$par.ests[3], mod.t3$par.ests[1])
 # Combine simulated marginals
Sim_val.ln_gumb_t <- cbind(sim.val.ln1_gumb_t, sim.val.ln2_gumb_t, sim.val.ln3_gumb_t)

# Transform copula values to t-Student marginals for the Clayton copula
sim.val.ln1_clay_t <- qst(Sim_val.ln_clayC[, 1], mu = mod.t1$par.ests[2],
                          sd = mod.t1$par.ests[3], mod.t1$par.ests[1])
sim.val.ln2_clay_t <- qst(Sim_val.ln_clayC[, 2], mu = mod.t2$par.ests[2],
                          sd = mod.t2$par.ests[3], mod.t2$par.ests[1])
sim.val.ln3_clay_t <- qst(Sim_val.ln_clayC[, 3], mu = mod.t3$par.ests[2],
                          sd = mod.t3$par.ests[3], mod.t3$par.ests[1])
# Combine simulated marginals
Sim_val.ln_clay_t <- cbind(sim.val.ln1_clay_t, sim.val.ln2_clay_t, sim.val.ln3_clay_t)

# Calculate portfolio losses for each copula simulation
# Gumbel Copula
MC.Lsim_gumb_t <- -V * (Sim_val.ln_gumb_t %*% w)  # Portfolio losses for Gumbel copula
quant_gumbel_t <- quantile(MC.Lsim_gumb_t, alfas)  # Quantiles for Gumbel copula
mean_exceed_gumbel_t_99 <-
  mean(MC.Lsim_gumb_t[MC.Lsim_gumb_t > quant_gumbel_t[1]])  # Mean exceedance at 99%
mean_exceed_gumbel_t_995 <-
  mean(MC.Lsim_gumb_t[MC.Lsim_gumb_t > quant_gumbel_t[2]])  # Mean exceedance at 99.5%
mean_exceed_gumbel_t_999 <-
```

```r
  mean(MC.Lsim_gumb_t[MC.Lsim_gumb_t > quant_gumbel_t[3]])  # Mean exceedance at 99.9%

# Clayton Copula
MC.Lsim_clay_t <- -V * (Sim_val.ln_clay_t %*% w)  # Portfolio losses for Clayton copula
quant_clayton_t <- quantile(MC.Lsim_clay_t, alfas)  # Quantiles for Clayton copula
mean_exceed_clayton_t_99 <-
  mean(MC.Lsim_clay_t[MC.Lsim_clay_t > quant_clayton_t[1]])  # Mean exceedance at 99%
mean_exceed_clayton_t_995 <-
  mean(MC.Lsim_clay_t[MC.Lsim_clay_t > quant_clayton_t[2]])  # Mean exceedance at 99.5%
mean_exceed_clayton_t_999 <-
  mean(MC.Lsim_clay_t[MC.Lsim_clay_t > quant_clayton_t[3]])  # Mean exceedance at 99.9%

# Create a results data frame for both copulas
results <- data.frame(
  Copula = c("Gumbel", "Clayton"),
  VAR_99 = c(quant_gumbel_t[1], quant_clayton_t[1]),
  VAR_995 = c(quant_gumbel_t[2], quant_clayton_t[2]),
  VAR_999 = c(quant_gumbel_t[3], quant_clayton_t[3]),
  ES_99 = c(mean_exceed_gumbel_t_99, mean_exceed_clayton_t_99),
  ES_995 = c(mean_exceed_gumbel_t_995, mean_exceed_clayton_t_995),
  ES_999 = c(mean_exceed_gumbel_t_999, mean_exceed_clayton_t_999)
)

# Print the final results as a table
kable(results)


############################
## Extreme Value Theory ##
############################


# Historical Simulation for Loss Analysis

# Compute portfolio losses as negative weighted log-returns
SH.L <- -(val.ln %*% w)  # Portfolio losses
L_pos <- SH.L[SH.L > 0]  # Filter positive losses
L_pos <- L_pos[order(L_pos)]  # Order positive losses in ascending order
np <- length(L_pos)  # Total number of positive losses
q <- 0.999  # Quantile level for VaR estimation

# Define x-axis values for density plots
x <- seq(0, max(L_pos), length = 2000)  # Range of loss sizes
x0.L <- rep(0, length(L_pos))  # Baseline for plotting points

# Fit Weibull distribution to the positive losses
par.W <- fitdistr(L_pos, "weibull")  # Estimate Weibull parameters
AIC_W <- AIC(par.W)  # Calculate AIC for Weibull model
Den.Weibull.L <-
  dweibull(x, shape = par.W$estimate[1], scale = par.W$estimate[2])
VaR_Weibull <-
  qweibull(q, shape = par.W$estimate[1], scale = par.W$estimate[2])
# Fit Log-normal distribution to the positive losses
```

```r
par.Ln <- fitdistr(L_pos, "log-normal")  # Estimate Log-normal parameters
AIC_Ln <- AIC(par.Ln)  # Calculate AIC for Log-normal model
Den.ln.L <-
  dlnorm(x, meanlog = par.Ln$estimate[1], sdlog = par.Ln$estimate[2])
VaR_Ln <-
  qlnorm(q, meanlog = par.Ln$estimate[1], sdlog = par.Ln$estimate[2])

# Plot density functions for Weibull and Log-normal distributions
plot(
  x, Den.Weibull.L, type = "l", col = "blue",
  xlab = "Loss Size", ylab = "Density",
  main = "Tail Distribution Comparison", ylim = c(0, max(c(Den.Weibull.L, Den.ln.L)))
)
lines(x, Den.ln.L, col = "green")  # Add Log-normal density line
points(L_pos, rep(0, length(L_pos)), col = "black", pch = 16, cex = 0.5)
legend(
  "topright", legend = c("Weibull", "Log-normal"),
  col = c("blue", "green"), lty = 1
)  # Add legend

# Create a data frame to compare AIC and VaR values for the two distributions
AIC_VaR_values <- data.frame(
  Distribution = c("Weibull", "Log-normal"),
  AIC = c(AIC_W, AIC_Ln),  # Akaike Information Criterion
  VaR_99_9 = c(VaR_Weibull, VaR_Ln)  # Value at Risk at 99.9% confidence level
)

# Display the comparison table
kable(AIC_VaR_values)

# Extreme Value Analysis with Pareto and GPD Models

# Define quantile level and use the sorted losses
q <- 0.999  # Confidence level for VaR estimation
L <- L_pos  # Positive losses from historical simulation
n <- length(L_pos)  # Total number of positive losses

# Sort losses in descending order and select the largest 50 observations
L <- sort(L, decreasing = TRUE)
L50 <- L[1:50]  # Largest 50 losses

# Fit Pareto distribution to the top 50 losses
epar <- epareto(L50, method = "mle")  # Maximum likelihood estimation for Pareto
k <- epar$parameters[1]  # Scale parameter for Pareto
Hpar <- epar$parameters[2]  # Shape parameter for Pareto
fr <- (n - 50) / n  # Exceedance fraction
alpha <- q - fr  # Adjusted confidence level
VaR_Par <- k / ((1 - alpha / (1 - fr))^(1 / Hpar))  # VaR at 99.9% confidence for Pareto

# Fit Generalized Pareto Distribution (GPD) to the full dataset above the threshold
threshold <- L50[50]  # Threshold at the 50th largest loss
par_GPD <- gpd(L, threshold = threshold)  # Fit GPD model
fr <- (n - 50) / n  # Exceedance fraction for GPD
```

```r
conf <- q - fr  # Adjusted confidence level for GPD
VaR_GPD <- evir::qgpd(conf / (1 - fr), par_GPD$par.ests[1], 0,
                      par_GPD$par.ests[2]) + threshold  # VaR at 99.9% confidence for GPD

# Generate diagnostic visualizations for extreme value analysis
par(mfrow = c(1, 2))  # Set layout for two side-by-side plots
hp <- hillplot(L_pos)  # Generate Hill plot
meplot(L_pos)  # Generate Mean Excess plot
par(mfrow = c(1, 1))  # Reset layout to single plot
cvplot(L_pos)  # Generate Coefficient of Variation plot

# Summarize results into a comparison table
results <- data.frame(
  Model = c("Pareto", "GPD"),  # Model names
  Scale = c(k, par_GPD$par.ests[1]),  # Scale parameters
  Shape = c(Hpar, NA),  # Shape parameters (NA for GPD)
  Threshold = c(NA, threshold),  # Thresholds (only for GPD)
  VaR_99_9 = c(VaR_Par, VaR_GPD)  # VaR at 99.9% confidence level
)

# Print results as a table
kable(results)
```